



# Bug Bounty Report

**LoadingZone.depositWithPermit: Missing Access  
Control Allows Permit Front Running and Forces an  
Early Deposit**

*alva-p.xyz*

August 2026

---

Prepared by: alva-p

Lead Researcher: Pineda Alvaro

## Summary

`LoadingZone.depositWithPermit()` accepts an ERC-2612 signature from any external caller. Anyone who observes a user's permit can submit it before the intended transaction, consume the permit nonce, and force the user's deposit to execute earlier than intended. The victim's funds remain credited to the victim, so the issue causes timing manipulation and griefing rather than theft.

**Severity:** Low

**Affected code:** `src/core/balance>LoadingZone.sol`, lines 121 to 132

## Root Cause

The function binds the permit owner and credited balance to `user`, but does not bind the transaction caller:

```
1 function depositWithPermit(  
2     address user,  
3     uint256 amount,  
4     uint256 deadline,  
5     uint8 v,  
6     bytes32 r,  
7     bytes32 s  
8 ) external {  
9     require(amount > 0, ...);  
10    IERC20Permit(address(depositToken)).permit(  
11        user, address(this), amount, deadline, v, r, s  
12    );  
13    require(depositToken.transferFrom(user, address(this), amount),  
14        ...);  
14    balances[user] += amount;  
15    emit TokenDeposited(user, amount, balances[user]);  
16 }
```

The related `depositForWithPermit()` path restricts its caller to `depositManager`, and its NatSpec explicitly recognizes that a permissionless permit consumer can replay an observed signature or submit it first. `depositWithPermit()` lacks the equivalent gate.

---

## Happy Path vs. Attack Path

### Happy Path

1. A user signs a permit for amount `X` and a chosen deadline.
2. The intended user or TipRun flow submits the signature at the desired time.
3. The token consumes the nonce, transfers `X`, and the LoadingZone credits `balances[user]`.

### Attack Path

1. User `U` prepares a permit for amount `X`.
2. `U` broadcasts the transaction or sends the signature through an observable external channel.
3. Griever `G` copies the permit and calls `depositWithPermit(U, X, deadline, v, r, s)` first.
4. `U`'s tokens immediately move into LoadingZone and are credited to `balances[U]`.
5. The ERC-2612 nonce is consumed.
6. The intended transaction reverts because its signature nonce is no longer valid.

## Impact

The attacker controls when the victim's otherwise valid deposit executes. The victim's permit nonce is consumed before the intended transaction, which then reverts and wastes gas. Deposit workflows that depend on timing can be disrupted and require a fresh signature.

There is no direct theft: the deposited tokens are always credited to the permit signer.

## Proof of Concept

The Foundry reproduction is located at:

```
1 findings/M-01-DepositWithPermitFrontRun/test.t.sol
```

It demonstrates that an unrelated caller can submit the victim's valid permit, transfer the victim's tokens into LoadingZone, consume the nonce, and cause reuse of the same signature to revert.

---

## Fix Direction

Bind execution to the user:

```
1  function depositWithPermit(...) external {  
2  +    require(msg.sender == user, "LoadingZone: caller must be user"  
3      );  
4      ...  
5  }
```

If the intended product flow is relayed rather than submitted directly by the user, restrict the function to `depositManager`, matching `depositForWithPermit()`.

## Report Boundary

This report does not claim token theft. The attacker cannot redirect the deposit or change the credited account; the demonstrated impact is forced timing, nonce consumption, failed execution of the victim's original transaction, and associated grieving.